

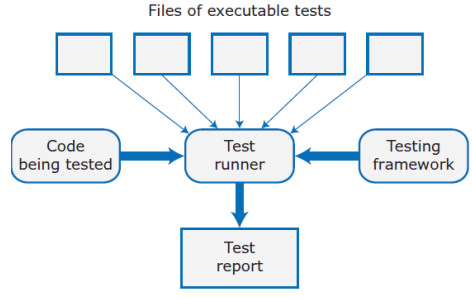
07) – Automação de Testes

Prof. Ivan Granja
Disciplina: Verificação e validação de Software

1

Automação de Testes

- Inovação e contribuição das mais significativas da engenharia de software, depois do advento dos métodos ágeis, **é o teste automatizado.**
- Amplamente utilizado em empresas de desenvolvimento de software, baseia-se na ideia de que os testes devem ser executáveis.



2

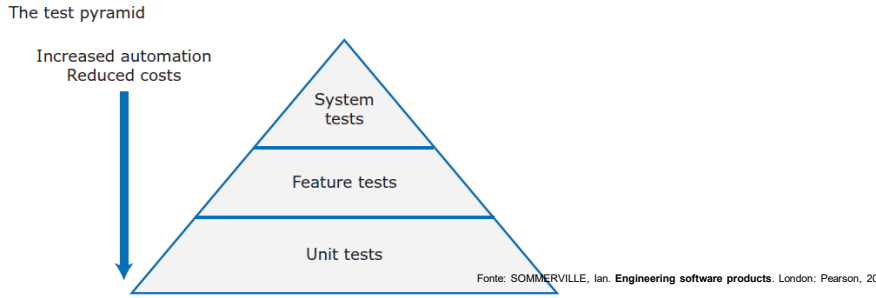
Automação de Testes

- Um teste executável inclui os dados de entrada requeridos pelo unidade/objeto que em teste, o resultado esperado e uma verificação do que a unidade retorna contra o resultado esperado. Você executa o teste e ele é aprovado se a unidade retornar o resultado esperado.

- Normalmente, é possível desenvolver centenas ou milhares de testes executáveis (a maioria unitário) para um produto de software, pois o custo de desenvolivmento não é muito alto na base da pirâmide do teste.

3

Lembra da 1ª Aula? **A Pirâmide do Teste**



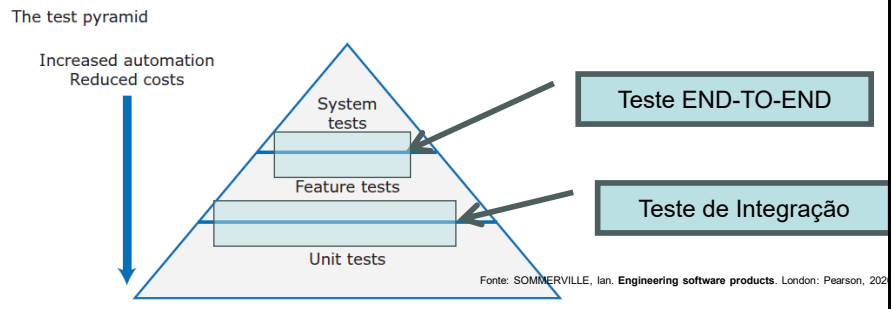
O custo da automação de teste aumenta conforme nos afastamos da base da pirâmide

- Teste Unitários
- Testes de Integração (entre APIs e serviços externos)
- Teste de *features* (‘pedaços’ ou grandes funcionalidades)
- Teste E-2-E e Teste do Sistema

4

Lembra da 1ª Aula? A Pirâmide do Teste

5



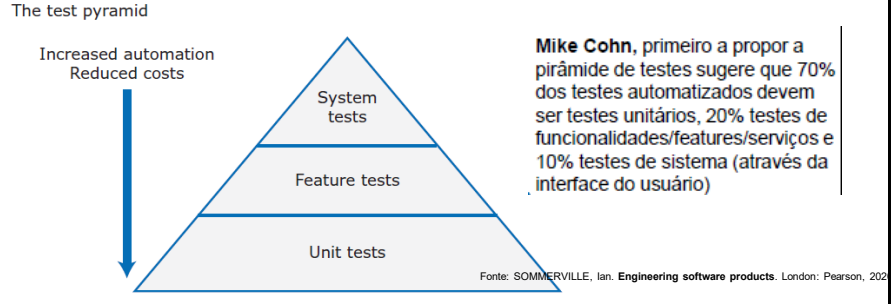
O custo da automação de teste aumenta conforme nos afastamos da base da pirâmide

- Teste Unitários
- Testes de Integração (entre APIs e serviços externos)
- Teste de *features* (‘pedaços’ ou grandes funcionalidades)
- Teste E-2-E e Teste do Sistema

5

Lembra da 1ª Aula? **A Pirâmide do Teste**

6



O custo da automação de teste aumenta conforme nos afastamos da base da pirâmide

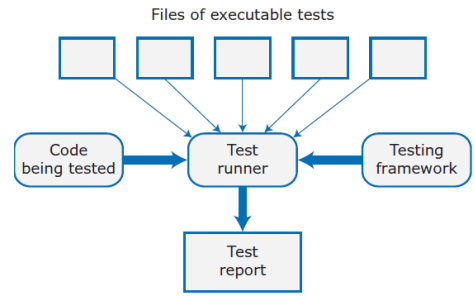
- Teste Unitários
- Testes de Integração (entre APIs e serviços externos)
- Teste de *features* (‘pedaços’ ou grandes funcionalidades)
- Teste E-2-E e Teste do Sistema

6

PUC CAMPINAS

Frameworks de Automação de Testes

- Testes automatizados são implementados como uma subclasse da classe TestCase para teste o objeto de teste alvo.
- Os frameworks de teste incluem uma maneira de executar o objeto (**TestRunner**) e então verificar e reportar os resultados dos testes (***assert methods***).



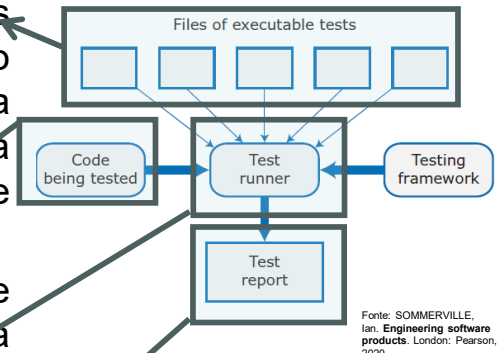
Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 2020.

7

PUC CAMPINAS

Frameworks de Automação de Testes

- testes automatizados são implementado como uma subclasse da classe TestCase para teste o objeto de teste alvo.
- Os frameworks de teste incluem uma maneira de executar o objeto (**TestRunner**) e então verificar e reportar os resultados dos testes (***assert methods***).



Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 2020.

Para entender e depois generalizar entendimento do que é a automação de teste, começaremos pelos testes mais simples, que são os Teste Unitários (teste do DEV)

8

Que estrutura um framework propicia para construir um teste automatizado?

- Os frameworks de teste fornecem uma classe base e pronta, em geral chamada "TestCase“ ou similar, que é usada pelo **framework** de teste.
- Para criar um teste automatizado, o *tester* define sua própria classe de teste como uma subclasse (herança) desta classe TestCase do framework e escreve testes (*scripts*) como métodos desta sua classe.
- Os frameworks de teste incluem uma maneira de executar estes testes (**TestRunner**) e então verificar e reportar os resultados dos testes (***assert methods***).

Exemplo de estrutura de um **Teste Unitário** automatizado

- **Lembre**: antes de automatizar, prepare o teste....
  - Aplicação das técnicas, para definição da massa de dados, variações de casos de teste e da saída esperada (Oráculo)
  - **Sem usar técnicas, o teste será aleatório**, o que exigirá muitas combinações para ser eficaz. Nesta condições o custo do Oráculo tende a ser **CARÍSSIMO**!!!!!
- Estrutura-se o **código de um teste automatizado** (é um ‘script’, lembrar sempre...) **com três partes:**

1. **Preparar (*Arrange*):** configurar o sistema e as condições no *script* p/ executar o teste, envolvendo:
   - Definir os parâmetros e condições de teste (instanciação de objetos, definição de constantes e variáveis, etc..)
   - Se necessário, preparar objetos simulados que emulam (*mocking*) a funcionalidade de um código que ainda não foi desenvolvido (ou não é objeto principal do teste de agora).

Exemplo de estrutura de um **Teste Unitário** automatizado

2. **Executar (*Action*):** Chamar a unidade (objeto de teste) que está sendo testada com os parâmetros (dados) de teste

- Aciona-se o método que será testado – ou função, ou serviço, etc... –, passando-lhe os dados definidos na preparação do teste e recolhendo e armazenando a saída real produzida.

3. **Verificar (*Assert*):** Checagem sobre o que deveria acontecer (**saída esperada**) contra aquilo que a unidade (objeto de teste) que está sendo testada produziu (**saída real)** depois de executada.

- Usa-se algum **Assert Method**, que define se o resultado do teste: **PASS** ou **FAIL**

Exemplo de **Teste Unitário** Automatizado

O código exemplo mostra um teste automatizado simples (em Python) para uma função que faz cálculos de planejamento financeiro

A função (em Python) **interest_calculator (p, r, n)** calcula os juros devidos com base no valor e no prazo do empréstimo.

Classe TestCase herdada do framework

```
# TestInterestCalculator inherits attributes and methods from the class
# TestCase in the testing framework unittest
class TestInterestCalculator (unittest.TestCase):

        # Define a set of unit tests where each test tests one thing only
        # Tests should start with test_ and the name should explain
        # what is being tested

        def test_zeroprincipal (self):

                #Arrange - set up the test parameters
                p = 0
                r = 3
                n = 31
                result_should_be = 0

                #Action - Call the method to be tested
                interest = interest_calculator (p, r, n)

                #Assert - test what should be true
                self.assertEqual (result_should_be, interest)
```

Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 2020.

Exemplo de **Teste Unitário** Automatizado

O código exemplo mostra um teste automatizado simples (em Python) para uma função que faz cálculos de planejamento financeiro

A função (em Python) **interest_calculator (p, r, n)** calcula os juros devidos com base no valor e no prazo do empréstimo.

Código do Caso de Teste (uma automação do teste)
Em geral, prepara-se mais de uma

```
# TestInterestCalculator inherits attributes and methods from the class
# TestCase in the testing framework unittest

class TestInterestCalculator (unittest.TestCase):

        # Define a set of unit tests where each test tests one thing only
        # Tests should start with test_ and the name should explain
        # what is being tested

        def test_zeroprincipal (self):

                #Arrange - set up the test parameters
                p = 0
                r = 3
                n = 31
                result_should_be = 0

                #Action - Call the method to be tested
                interest = interest_calculator (p, r, n)

                #Assert - test what should be true
                self.assertEqual (result_should_be, interest)
```

Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 2020.

13

Exemplo de **Teste Unitário** Automatizado

O código exemplo mostra um teste automatizado simples (em Python) para uma função que faz cálculos de planejamento financeiro

A função (em Python) **interest_calculator (p, r, n)** calcula os juros devidos com base no valor e no prazo do empréstimo.

Arrange (Preparar)

```
# TestInterestCalculator inherits attributes and methods from the class
# TestCase in the testing framework unittest

class TestInterestCalculator (unittest.TestCase):

        # Define a set of unit tests where each test tests one thing only
        # Tests should start with test_ and the name should explain
        # what is being tested

        def test_zeroprincipal (self):

                #Arrange - set up the test parameters
                p = 0
                r = 3
                n = 31
                result_should_be = 0

                #Action - Call the method to be tested
                interest = interest_calculator (p, r, n)

                #Assert - test what should be true
                self.assertEqual (result_should_be, interest)
```

Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 2020.

14

Exemplo de **Teste Unitário** Automatizado

15

O código exemplo mostra um teste automatizado simples (em Python) para uma função que faz cálculos de planejamento financeiro

A função (em Python) **interest_calculator (p, r, n)** calcula os juros devidos com base no valor e no prazo do empréstimo.

Action (Executar)

```
# TestInterestCalculator inherits attributes and methods from the class
# TestCase in the testing framework unittest

class TestInterestCalculator (unittest.TestCase):

        # Define a set of unit tests where each test tests one thing only
        # Tests should start with test_ and the name should explain
        # what is being tested

        def test_zeroprincipal (self):

                #Arrange - set up the test parameters
                p = 0
                r = 3
                n = 31
                result_should_be = 0

                #Action - Call the method to be tested
                interest = interest_calculator (p, r, n)

                #Assert - test what should be true
                self.assertEqual (result_should_be, interest)
```

Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 2020.

15

Exemplo de **Teste Unitário** Automatizado

16

O código exemplo mostra um teste automatizado simples (em Python) para uma função que faz cálculos de planejamento financeiro

A função (em Python) **interest_calculator (p, r, n)** calcula os juros devidos com base no valor e no prazo do empréstimo.

Assert (verificar)

```
# TestInterestCalculator inherits attributes and methods from the class
# TestCase in the testing framework unittest

class TestInterestCalculator (unittest.TestCase):

        # Define a set of unit tests where each test tests one thing only
        # Tests should start with test_ and the name should explain
        # what is being tested

        def test_zeroprincipal (self):

                #Arrange - set up the test parameters
                p = 0
                r = 3
                n = 31
                result_should_be = 0

                #Action - Call the method to be tested
                interest = interest_calculator (p, r, n)

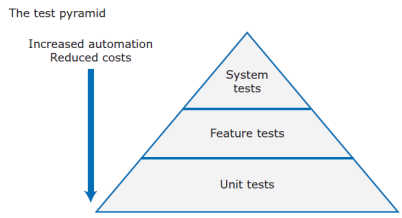
                #Assert - test what should be true
                self.assertEqual (result_should_be, interest)
```

Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 2020.

16

Lembra da 1ª Aula? **A Pirâmide do Teste**
Teste de Integração e de features

- A implementação de funcionalidades ou *features* em software envolve a integração de unidades funcionais (pacotes, classes, componentes, serviços.)
  - E como estas interagem....



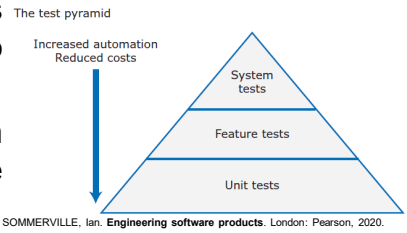
Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 2020.

- Bons testes unitários, dão maior confiança de que as unidades funcionais e os componentes que a implementam se comportarão, individualmente, conforme o esperado.
- Unidades, partes e componentes podem até funcionar bem separadamente; mas, também podem interagir entre si de forma inesperada quando postas juntas; portanto, ainda **será necessário fazer testes de integração** e **testes de features e o end-to-end** (grandes funcionalidades, processos, etc.)
  - Estes Teste 'provocam' a integração das unidades acionadas, mas estas são melhor verificadas dois a dois (pares)

17

Lembra da 1ª Aula? **A Pirâmide do Teste**
Teste de Integração e de features

- Usuários acessam funcionalidades por meio da interface gráfica do (**GUI**) do produto.
- No entanto, o teste baseado em **GUI** é caro para automatizar e gerenciar;
  - **Há estratégia alternativa** para fazer o teste da feature.



Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 2020.

- Isso envolve projetar o software para partes implementadas serem acessadas por meio de API, e não apenas pela **GUI**.
  - **Decacoplar o FrontEnd (GUI) do BackEnd (APIs/serviços)**
- Parte dos testes de funcionalidade/integração podem avaliar as funcionalidades diretamente por meio das API (20%- **Conh**)
- Outra parte por meio da interação direta do usuário por meio da interafce como o sistema (10% - segundo **Conh**)

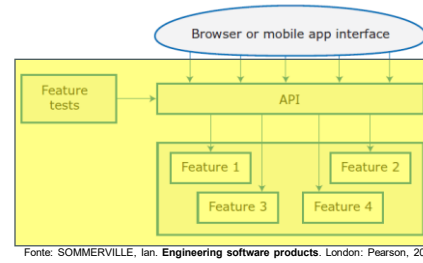
18

Testando com ou sem interação com a GUI
Teste de Integração e de features

19

- A Figura ao lado mostra como se comporta o teste de features/integração diretamente por meio da API, sem interação direta do usuário por meio da interface do sistema (**GUI**), seja browser ou mobile APP.

Feature testing through an API



Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 202

- Por exemplo, uma série de chamadas de API pode ser usada para **'emular' a mesma sequência** que ocorre na tela da feature do sistema. Ex: para compartilhar um documento com outro usuário, especificando seu e-mail:
  1) Coletar o endereço de e-mail do outro usuário
  2) Coletar as informações de identificação do documento
  3) Verificar se as permissões de acesso permitem o compartilhamento
  4) Verificar se o e-mail especificado é válido e pertence a um usuário registrado.
  5) Adicionar o documento ao espaço de trabalho compartilhado

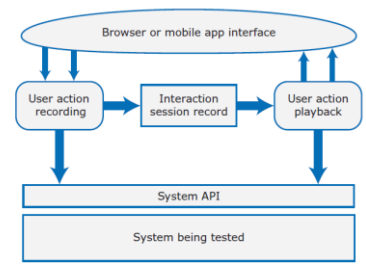
19

Testes End-to-End (grandes funções) pela GUI
Também para testes de sistema

20

- Esta outra Figura mostra o teste usando a **GUI** do browser ou o app mobile, por interação direta com a interface do sistema.
- Não é simples implementar testes automatizados End-to-End e de funcionalidades pela **GUI:**
  - Ferramentas caras
  - Várias verificações de resultado (***asserts)*** são necessárias – memorize esta a palavra)
  - Assim, exige checar se cada umas das etapas do teste foram executadas conforme o esperado.
- Alguma automação de testes de funcionalidades pode ser possível usando uma estrutura parecida com testes unitários
  - Na maioria das vezes é necessário usar uma ferramenta (ou *framework*) especializada para este tipo de testes.

Interaction recording and playback



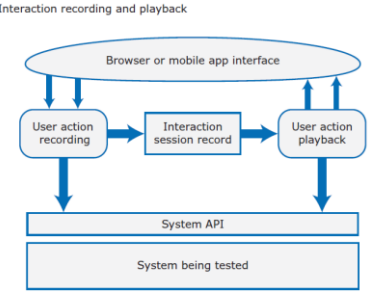
Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 202

20

Testes End-to-End (grandes funções) pela GUI
Também para testes de sistema

21

- Testes de sistema e End-to-End também servem para verificar os dois tipos de requisitos:
  - **RFs** (Grandes Funções do sistema e suas Regras de Negócio) – checando se a saída está correta e o comportamento da tela também....
  - **RNFs** (Não-Funcionais), como *timeout*, performance, segurança...



Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 202

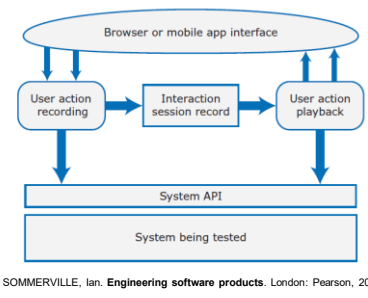
- O testador de sistema deve definir um caso de teste para cada comportamento importante suportado pela tela (seja de um ou vários passos), por seleção de itens em menus, de telas, inserção de informações pelo teclado, *clicks* em botões, etc.
  - Identificar pontos onde uma verificação intemediária (*assert*) do resultado até aquele ponto do processo será necessária.
  - Definir as saídas esperadas (Oráculo)

21

Testes End-to-End (grandes funções) pela GUI
Também para testes de sistema

22

- Testes de Sistema e End-to-End envolvem testar como se fosse o usuário, identificar como este usa a **GUI** que está sendo testada - os cenários comportamentais. E então executá-los como testes:
  - Manuais – Roteiros em "excel"
  - Automatizados – por meio de ferramenta ou *framework*



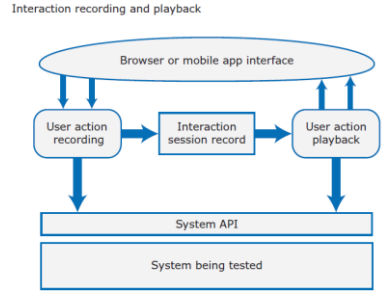
Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 202

- Deve-se buscar comportamentos (sequencias/interações na tela) e entre diversas telas que possam causar problemas, ou sejam, que levem a falhas do sistema e a outras anomalias.
- **End-to-End** é mais do que testar uma funcionalidade grande (que engloba várias features). Pode ser também:
  - Um processo de negócio (prestação de contas), um ciclo de vida do negócio (abrir, operar e fechar uma conta de banco)

22

Testes End-to-End (grandes funções) pela GUI
Também para **testes de sistema**

- As ferramentas de gravação de interação guardam movimentos e cliques, seleções de menu, entradas de teclado e as registram.
- Salvam a sessão de interação e podem reproduzi-la depois (*playback*), enviando comandos para o aplicativo e replicando-os na interface do usuário (GUI).



Fonte: SOMMERVILLE, Ian. **Engineering software products**. London: Pearson, 202

- Essas ferramentas também oferecem suporte ao testador programar 'scripts' de automação, para que possa escrever e executar estes cenários expressos como automação de teste.
- Ex: Isso é particularmente útil para testes de usabilidade, de múltiplos browsers ou SOs, pois ajuda a verificar se o seu software funciona da mesma maneira em diferentes navegadores ou Sistemas operacionais *mobile*.

Principais Frameworks de Automação de Testes

- Os frameworks de teste fornecem uma classe base e pronta, em geral chamada "TestCase", que é usada pelo **framework** de teste.
  - Teste Unitário: **JUnit, PyTest, PyUnit, NUnit**
  - Teste API ou BDD: **Postman, Cucumber**
  - Teste de Features, Sistema e End-To-End: **Selenium**, **Cypres, Playwright**
- Para criar um teste automatizado, o *tester* define sua própria classe de teste como uma subclasse (herança) desta classe TestCase do framework.
- Os frameworks de teste incluem uma maneira de executar (**TestRunner**) e então verificar e reportar os resultados dos testes (***assert methods***).